QI TECH

BOOT CAMP

HACKATHON QI TECH 2026 · RISK SOLUTIONS

Motor Antifraude Pix

Grupo 12 · RNV — README do projeto

TIME

Ramon Guimarães Cruz

Victoria Camile Bertuchi

Natalia Lugão

HACKATHON QI TECH 2026 · RISK SOLUTIONS

Grupo 12 - RNV

Integrantes: Ramon Guimarães, Victoria Camile e Natalia Lugão

SUMÁRIO

Como o motor de decisão funciona 1
Regras 2
Como as regras viram decisão 8
Como rodar e testar 9
Modelo de dados 11
Opções descartadas 12
Limitações conhecidas 12
Contrato da API 13

Como o motor de decisão funciona

O motor antifraude de Pix do RNV analisa requisições em `POST /pix/transaction` e decide entre `approved`, `two_factor_required` ou `rejected`, retornando uma justificativa explicável em `metadata`. A análise é imutável e consultável via `GET /pix/transaction/{id}`. Trata-se de um serviço interno chamado exclusivamente por outros microsserviços do RNV (via header `INTERNAL-TOKEN: internal-token-dev`).

A Tese dos 3 Eixos: Qual é a Dúvida?

Em vez de um score de probabilidade opaco, a política decompõe o risco em três eixos ortogonais:

Eixo	Pergunta Central	O 2FA Resolve?	Diretriz
WHO (dispositivo, canal, rede)	*É o titular legítimo operando?*	**Sim**	Dúvida de identidade: solicitar segundo fator (2FA)
HOW (valor, horário, velocity)	*O comportamento é habitual do pagador?*	**Sim**	Desvio de hábito: solicitar confirmação forte (2FA)
WHERE (conta e instituição de destino)	*O dinheiro vai para conta-laranja ou mula?*	**NÃO**	Em golpe ou coação, a vítima aprova o 2FA. Requer **bloqueio imediato** (`rejected`)

Arquitetura de Alto Desempenho

- **Camadas Estritas:** `Resource` (validação de formato JSON Schema na porta HTTP) → `Controller` (validação semântica de CPF/CNPJ, datas e regras negociais) → `Repository` (consultas otimizadas cobrindo índices de janela de 90 dias) → `Model`. A integridade das camadas é garantida por testes de AST (`tests/architecture/`).
- **Motor Puro:** A função `evaluate(RiskContext) -> EngineResult` em `src/engine/` é puramente matemática (zero I/O, determinística, sem acesso a banco ou relógio), permitindo testes

instantâneos e versionamento por `POLICY_VERSION`.

- **Idempotência Atômica:** Chave natural única `transaction_id` com SHA-256 do payload canônico. Retentativas idênticas devolvem `200` com a mesma análise reconstruída via DTO; payloads conflitantes com o mesmo ID retornam `422 RNV001024`.

Regras

A tabela a seguir consolida todas as 17 regras implementadas no código, com os parâmetros exatos definidos em `src/engine/policy.py` e suas justificativas regulatórias ou de design:

Regra	Lógica	Parâmetros	Por quê
`WHO_DEVICE_COMPROMISED`	Eixo `who`, `critical`. Dispara quando o device do Pix já foi usado por **3 ou mais titulares distintos** no total (contados por `account_holder`, ou seja, por documento) **ou** por **2 ou mais titulares distintos nas últimas 24 horas**; basta um dos dois gatilhos. A evidência vem de outra entidade (o device), e não da falta de histórico do pagador; por isso pode rejeitar a 1ª transação do cliente (`critical_axis`). Vale para todos os fluxos.	`DEVICE_COMPROMISED_DOCUMENTS_TOTAL` = 3 `DEVICE_COMPROMISED_DOCUMENTS_24H` = 2	`DEVICE_COMPROMISED_DOCUMENTS_TOTAL`: escolha de design, sem norma por trás. Um aparelho compartilhado por vários titulares é o sinal clássico de aparelho de fraude ou de mula; o corte em 3 deixa de fora o uso legítimo de 2 titulares no mesmo aparelho (por exemplo, um casal), que é comum. `DEVICE_COMPROMISED_DOCUMENTS_24H`: escolha de design. Uma rajada recente de titulares diferentes no mesmo aparelho é mais grave do que o acúmulo ao longo do tempo, por isso o corte é menor (2). A janela de 24h é a mesma do fan-in do destino. É `critical` e não `suspicious` porque é dúvida de quem está usando o aparelho que o 2FA não resolve: o 2FA seria aprovado no próprio aparelho comprometido. Não foi calibrado ainda no simulador.
`HOW_NIGHT_LIMIT_EXCEEDED`	Eixo `how`, `suspicious`. Pix PF→PF entre titulares diferentes, feito no período noturno local do pagador, cuja soma (Pix anteriores aprovados ou em 2FA no mesmo período noturno, mais o Pix em análise) fica estritamente acima do limite. Pix às 02h soma desde as 20h do dia anterior. Self-Pix e fluxos PF→PJ, PJ→PF e PJ→PJ ficam de fora.	`NIGHT_START_HOUR` = 20h `NIGHT_END_HOUR` = 6h `NIGHT_LIMIT_MINOR[pf_pf]` = R$ 1.000	`NIGHT_START_HOUR`, `NIGHT_END_HOUR`: janela noturna do Pix entre pessoas físicas, âncora regulatória do BCB (número da norma não citado: fonte primária ainda não conferida pelo time). `NIGHT_LIMIT_MINOR[pf_pf]`: limite noturno regulatório entre pessoas físicas, o mesmo valor do piso `FLOOR_NIGHT_LIMIT`; estritamente acima, então R$ 1.000,00 exatos passam. Só vale para PF→PF porque o limite trata de pagamento entre pessoas; comércio noturno (PF→PJ) e folha (PJ) são normais, e o self-Pix não é pagamento a terceiro.
`HOW_VELOCITY_EXCEEDED`	Eixo `how`. Dispara pelo número de tentativas do pagador nos últimos 10 minutos, **contando todos os status** (`approved`, `two_factor_required` e `rejected`: insistência é sinal).	`VELOCITY_WINDOW_MINUTES` = 10 `VELOCITY_10MIN[pf_pf].suspicious` = 3 `VELOCITY_10MIN[pf_pf].c`	`VELOCITY_WINDOW_MINUTES`: escolha de design; 10 minutos é curto o bastante para pegar rajada e longo o bastante para uma pessoa legítima não chegar ao limite por

Regra	Lógica	Parâmetros	Por quê
	Chegou ao limite `suspicious`, a regra marca `suspicious`; chegou ao limite `critical`, marca `critical`. Os limites são por fluxo: PF paga em poucas tentativas, PJ envia em lote (folha de pagamento).	`ritical` = 6 `VELOCITY_10MIN[pf_pj].suspicious` = 3 `VELOCITY_10MIN[pf_pj].critical` = 6 `VELOCITY_10MIN[pj_pf].suspicious` = 20 `VELOCITY_10MIN[pj_pf].critical` = 40 `VELOCITY_10MIN[pj_pj].suspicious` = 20 `VELOCITY_10MIN[pj_pj].critical` = 40	acaso. `VELOCITY_10MIN` para PF: escolha de design, sem norma por trás. Uma pessoa física raramente faz 3 Pix em 10 minutos, então 3 já pede 2FA (dúvida de comportamento, que o 2FA resolve); 6 é o dobro e indica automação, por isso `critical`. `VELOCITY_10MIN` para PJ: 20 e 40, porque empresa paga em lote (folha de pagamento) e o corte de PF geraria falso positivo; o `critical` é o dobro do `suspicious`, a mesma proporção do PF (decisão D4 do time). Não foi calibrado ainda no simulador.
`FLOOR_UNREGISTERED_DEVICE`	Piso regulatório (eixo `floor`, sem nível): pagador PF com device sem histórico (`unknown`) dispara por **dois gatilhos**: (1) valor do Pix acima do limite por transação; ou (2) soma do dia-calendário local do pagador naquele device (Pix anteriores `approved` e `two_factor_required`, mais o Pix em análise) acima do limite diário. Garante no mínimo `two_factor_required` (`regulatory_floor`); sozinho nunca rejeita, a rejeição vinda de outros eixos prevalece, e a confiança não o remove.	`UNKNOWN_DEVICE_TX_LIMIT_MINOR[pf_pf]` = R$ 200 `UNKNOWN_DEVICE_TX_LIMIT_MINOR[pf_pj]` = R$ 200 `UNKNOWN_DEVICE_DAILY_LIMIT_MINOR[pf_pf]` = R$ 1.000 `UNKNOWN_DEVICE_DAILY_LIMIT_MINOR[pf_pj]` = R$ 1.000	`UNKNOWN_DEVICE_TX_LIMIT_MINOR`: limite por transação para dispositivo não cadastrado, parâmetro regulatório do Pix; estritamente acima, então R$ 200,00 exatos passam. `UNKNOWN_DEVICE_DAILY_LIMIT_MINOR`: limite diário para dispositivo não cadastrado, parâmetro regulatório do Pix; pega o fracionamento em vários Pix pequenos, e o "dia" é o dia-calendário local porque a regra é por dia, não por 24h corridas. Estritamente acima, então R$ 1.000,00 exatos passam. Os dois só valem com pagador PF (PF→PF e PF→PJ). Device `observing` ou `trusted` já tem histórico e não aciona o piso. O número da norma não é citado: a fonte primária ainda não foi conferida pelo time.
`FLOOR_NIGHT_LIMIT`	Piso regulatório (eixo `floor`, sem nível): PF→PF entre titulares diferentes, no período noturno local, com soma (incluindo o Pix em análise) estritamente acima do limite. Garante no mínimo `two_factor_required` (`regulatory_floor`); nunca suaviza um `rejected`, e a confiança não o remove.	`NIGHT_START_HOUR` = 20h `NIGHT_END_HOUR` = 6h `NIGHT_LIMIT_MINOR[pf_pf]` = R$ 1.000	Mesmos parâmetros e mesma fonte de `HOW_NIGHT_LIMIT_EXCEEDED`. A soma conta Pix `approved` e `two_factor_required` (tudo que pode ter saído). A regra do eixo `how` e o piso coexistem: o piso é a garantia regulatória, a regra é o sinal que entra na matriz.
`WHERE_DESTINATION_NEW_ACCOUNT`	Eixo `where`, `suspicious`. Dispara quando a conta de destino foi aberta há **menos** de 30 dias na data do Pix (estritamente abaixo: 29 dias dispara, 30 não). Vale para todos os fluxos.	`DESTINATION_NEW_ACCOUNT_DAYS` = 30 dias	`DESTINATION_NEW_ACCOUNT_DAYS`: escolha de design, sem norma por trás. Conta recém-aberta é o destino típico de golpe e de mula, e o 2FA não resolve dúvida de destino (eixo `where`). Os 30 dias são o mesmo corte de "destino jovem" usado em `WHERE_DESTINATION_FAN_IN` e no self-Pix maduro (`MATURE_SELF_PIX_MIN_DESTINATION_AGE_DAYS`), para as regras não se

Regra	Lógica	Parâmetros	Por quê
			contradizerem. Sozinha só pede 2FA; vira `rejected` quando outro eixo também está suspeito (matriz).
`WHERE_UNKNOWN_PARTICIPANT`	Eixo `where`, `suspicious`. Dispara quando o ISPB do participante de destino não está na Relação de participantes do STR carregada no banco (`listed = false`). Não é erro de entrada: o Pix é analisado normalmente e o participante é criado como não listado. Vale para todos os fluxos.	Sem parâmetro numérico. Referência: a lista do STR (BCB, 16/10/2026)	Destino fora da lista oficial é dúvida sobre o destino, que o 2FA não resolve, por isso é sinal e não rejeição direta: um ISPB legítimo recém-criado ou ausente da nossa cópia da lista não pode derrubar o Pix sozinho. Não há limiar a calibrar; o dado vem da tabela de referência `pix_participant`.
`WHERE_DESTINATION_FAN_IN`	Eixo `where`, `critical`. Conta de destino aberta há **menos** de 30 dias que recebeu de pelo menos N pagadores distintos nas últimas 24h, contando todas as tentativas (aprovadas, em 2FA e rejeitadas). N depende do fluxo. Destino com 30 dias ou mais fica de fora. Como a evidência vem do destino (outra entidade), pode rejeitar a 1ª transação do pagador.	`FAN_IN_CRITICAL_PAYERS_24H[pf_pf]` = 3 `FAN_IN_CRITICAL_PAYERS_24H[pj_pf]` = 3 `FAN_IN_CRITICAL_PAYERS_24H[pf_pj]` = 10 `FAN_IN_CRITICAL_PAYERS_24H[pj_pj]` = 10 `FAN_IN_WINDOW_HOURS` = 24 `DESTINATION_NEW_ACCOUNT_DAYS` = 30	`FAN_IN_CRITICAL_PAYERS_24H`: quando o destino é pessoa física, 3 pagadores distintos em 24h numa conta recém-aberta já é padrão de mula. Quando é pessoa jurídica, o limite sobe para 10 porque loja nova recebe de muitos clientes, mas uma falsa loja recebe de muitos mais. É `critical` porque o 2FA não resolve dúvida de destino. Os valores são escolha de design, sem norma por trás. `FAN_IN_WINDOW_HOURS`: a campanha de golpe aparece nas tentativas recentes, por isso a janela é curta e conta todos os status. `DESTINATION_NEW_ACCOUNT_DAYS`: o mesmo corte de `WHERE_DESTINATION_NEW_ACCOUNT`.
`WHERE_DESTINATION_REJECTIONS`	Eixo `where`, `critical`. Destino com rejeições de pelo menos 2 pagadores distintos nas últimas 24h (o mesmo pagador rejeitado várias vezes conta uma vez). Vale para todos os fluxos e para destino de qualquer idade. Como a evidência vem do destino (outra entidade), pode rejeitar a 1ª transação do pagador.	`DESTINATION_REJECTIONS_CRITICAL_PAYERS_24H` = 2 `FAN_IN_WINDOW_HOURS` = 24	`DESTINATION_REJECTIONS_CRITICAL_PAYERS_24H`: é a "fundada suspeita" da Res. BCB 501/2025, como registrado nas especificações do projeto (texto da norma ainda não conferido pelo time na fonte primária). Duas rejeições de pagadores diferentes tiram a dúvida de que seja um erro isolado de um cliente. Só conta `rejected`, não 2FA. `FAN_IN_WINDOW_HOURS`: a mesma janela de 24h do fan-in.

Regra	Lógica	Parâmetros	Por quê
`HOW_ABOVE_CUSTOMER_CEILING`	Eixo `how`, `suspicious`. Dispara quando o valor do Pix fica estritamente acima do **teto do cliente**. O teto mistura o histórico do pagador com um teto de referência: `teto = (n × teto_hist + K × teto_ref) / (n + K)`, com `n` = Pix aprovados do pagador nos últimos 90 dias e `K` = peso do teto de referência. `teto_hist` = `HISTORY_MARGIN` × percentil 95 (interpolação linear entre os valores ordenados) dos valores aprovados. `teto_ref` depende do pagador: PF com conta de origem aberta há menos de 90 dias, PF com 90 dias ou mais, ou PJ. Sem Pix aprovado (fase 0 do pagador), `n` = 0 e vale só o `teto_ref`: a ausência de histórico nunca é evidência, então a 1ª transação só dispara se passar do teto de referência. Conforme `n` cresce o teto passa a seguir o comportamento do próprio cliente (fases 1 e 2). O teto é truncado para centavos inteiros. Vale para todos os fluxos.	`REFERENCE_CEILING_NEW_ACCOUNT_DAYS` = 90 dias `REFERENCE_CEILING_MINOR.individual_new_account` = R$ 1.000 `REFERENCE_CEILING_MINOR.individual` = R$ 3.000 `REFERENCE_CEILING_MINOR.company` = R$ 20.000 `HISTORY_WINDOW_DAYS` = 90 dias `HISTORY_PERCENTILE` = 0,95 `HISTORY_MARGIN` = 1,5 `SHRINKAGE_K` = 5	`REFERENCE_CEILING_MINOR`: escolha de design, sem norma por trás (o R$ 1.000 não vem de resolução do BCB). É um ponto de partida conservador para quem ainda não tem histórico: conta de PF com menos de 90 dias tem teto menor porque a conta nova é o perfil de conta de passagem; PF com 90 dias ou mais tem R$ 3.000, acima do Pix de R$ 1.500 do payload do desafio, para a 1ª transação de um cliente comum não ser tocada; PJ tem R$ 20.000 porque pagamentos de empresa são maiores por natureza. Não foi calibrado ainda no simulador. `REFERENCE_CEILING_NEW_ACCOUNT_DAYS`: escolha de design; 90 dias é a janela de histórico da política, então uma conta que já completou a janela é tratada como estabelecida. `HISTORY_WINDOW_DAYS`: 90 dias é o limite de consulta no caminho síncrono (índice + intervalo de data), e cobre ciclos mensais de gasto. `HISTORY_PERCENTILE`: o p95 ignora o valor atípico raro do próprio cliente sem deixar o teto subir até o maior Pix já feito. `HISTORY_MARGIN`: 1,5 dá folga de 50% sobre o p95 para que variação normal de gasto não dispare; escolha de design, sem calibração ainda. `SHRINKAGE_K`: 5 equivale a dizer que o teto de referência pesa como 5 Pix aprovados; com poucos Pix o teto ainda é quase o de referência, com 5 aprovados já pesa meio a meio e depois o histórico passa a dominar. É o mesmo 5 que define a fase 2 do pagador (5 aprovados e 30 dias). Dúvida de comportamento (`how`) que o 2FA resolve, por isso `suspicious` e nunca `critical`.
`HOW_EARLY_MORNING_AMOUNT`	Eixo `how`, `suspicious`. Dispara quando o Pix é feito de madrugada no horário local do pagador (00:00 a 05:59, a hora local vem de `transaction_date` + `utc_offset_minutes`) e o valor está **estritamente acima** do mínimo e **até** o máximo da faixa (R$ 200,00 exatos não disparam; R$ 1.000,00 exatos disparam). Acima do máximo a regra não atua: PF→PF entre titulares diferentes já cai no limite noturno (`HOW_NIGHT_LIMIT_EXCEEDED`). Vale para todos os fluxos e	`EARLY_MORNING_START_HOUR` = 0h `EARLY_MORNING_END_HOUR` = 6h `EARLY_MORNING_MIN_AMOUNT_MINOR` = R$ 200 `EARLY_MORNING_MAX_AMOUNT_MINOR` = R$ 1.000	`EARLY_MORNING_START_HOUR`, `EARLY_MORNING_END_HOUR`: escolha de design, sem norma por trás; madrugada é a faixa em que o uso legítimo é raro, então um valor médio ali foge do padrão da maioria dos clientes (hipótese, ainda não medida no simulador); a janela é mais curta que a janela noturna regulatória (20h às 6h) de propósito, para o sinal ser mais específico. `EARLY_MORNING_MIN_AMOUNT_MINOR`: escolha de design; abaixo de R$

Regra	Lógica	Parâmetros	Por quê
	não depende de histórico do pagador (o sinal está no próprio Pix).		200 o dano é pequeno e o valor é comum em madrugada (táxi, lanche). Usa o mesmo número do limite de dispositivo não cadastrado só por coerência de ordem de grandeza, e tem parâmetro próprio para poder ser calibrado sozinho. `EARLY_MORNING_MAX_AMOUNT_MINOR`: escolha de design; R$ 1.000 é o limite noturno regulatório entre pessoas físicas, então acima disso o sinal já existe em outra regra e o piso regulatório cobre. É dúvida de comportamento (`how`) que o 2FA resolve, por isso `suspicious`. Não foi calibrado ainda no simulador.
`WHO_DEVICE_UNKNOWN`	Eixo `who`, `suspicious`. Dispara quando o device do Pix não tem nenhum Pix aprovado do pagador (estado `unknown`) **e** o pagador já tem pelo menos um outro device com Pix aprovado. Pagador sem nenhum device conhecido (1ª transação, ou só histórico sem aprovação) não dispara: ausência de histórico nunca é evidência, então o 1º device de um cliente não vira dúvida. O estado do device é `unknown` (sem aprovado), `observing` (aprovado, mas o primeiro aprovado tem 24h ou menos) ou `trusted` (aprovado há mais de 24h); só `unknown` dispara. É uma regra de sinal, independente do piso `FLOOR_UNREGISTERED_DEVICE`: o piso garante o 2FA por valor, esta regra entra na matriz e, somada a outro eixo suspeito, rejeita. Vale para todos os fluxos.	`DEVICE_UNKNOWN_MIN_KNOWN_DEVICES` = 1 `DEVICE_TRUSTED_AFTER_HOURS` = 24h	`DEVICE_UNKNOWN_MIN_KNOWN_DEVICES`: escolha de design. Um cliente que sempre usou outro aparelho e aparece num aparelho novo é dúvida de identidade (`who`), que o 2FA resolve; com 0 devices conhecidos não há contra o que comparar, por isso o mínimo é 1 e a 1ª transação fica `ok`. `DEVICE_TRUSTED_AFTER_HOURS`: escolha de design; um device aprovado há mais de 24h deixa de ser novo, e a espera impede que um fraudador ganhe confiança com um único Pix aprovado e use o device logo em seguida. Não foi calibrado ainda no simulador. Consequência para quem chama o serviço e para os cenários de teste: o mesmo pagador precisa mandar sempre o mesmo `device_id`. Trocar de device a cada Pix, depois de já ter histórico, dispara esta regra (um gerador de payload que sorteia um device novo por Pix faz um pagador legítimo parecer suspeito).
`WHO_AGGRAVATING_SIGNALS`	Eixo `who`, `suspicious`. Dispara quando ocorrem **2 ou mais sinais agravantes** do eixo `who` no Pix em análise. Agravantes avaliados: (1) IP com 5 ou mais pagadores distintos na última 1h (`ip.distinct_payers_1h`); (2) IP privado (`transaction.ip_is_private`: faixas RFC 1918, loopback e link-local; o IP do desafio 198.51.100.23 NÃO é privado); (3) canal novo para o pagador (`transaction.channel` fora de `known_channels`, avaliado apenas quando o pagador já possui histórico aprovado; ausência de histórico nunca é evidência). Um único agravante isolado mantém o eixo `wh`	`WHO_AGGRAVATING_MIN_SIGNALS` = 2 `WHO_AGGRAVATING_IP_DISTINCT_PAYERS_1H` = 5	`WHO_AGGRAVATING_MIN_SIGNALS`: escolha de design. Um único sinal técnico isolado (ex.: estar em Wi-Fi com IP privado ou em um quiosque/NAT compartilhado) pode ser legítimo; a combinação de 2 ou mais agravantes simultâneos remove a dúvida fortuita e indica risco de canal/rede, pedindo 2FA (dúvida de identidade, que o 2FA resolve). `WHO_AGGRAVATING_IP_DISTINCT_PAYERS_1H`: escolha de design; 5 pagadores diferentes num mesmo IP em 1h indica infraestrutura compartilhada atípica ou ataque distribuído.

Regra	Lógica	Parâmetros	Por quê
	`o` em `ok`; 2 ou mais agravantes marcam `suspicious`. Vale para todos os fluxos.		IP privado: RFC 1918 (10/8, 172.16/12, 192.168/16), loopback e link-local. O IP do desafio (198.51.100.23, TEST-NET) não é privado para evitar falso positivo no teste de referência. Canal novo: só avaliado para pagadores com histórico (fase 1 ou 2), garantindo que clientes em cold start não sofram penalização.
`HOW_RETRY_AFTER_REJECTION`	Eixo `how`, `suspicious`. É a "sondagem": o pagador teve um Pix `rejected` e, em **menos de** 10 minutos, tenta de novo com valor **estritamente menor** que o rejeitado. A rejeição considerada é a mais recente do mesmo pagador (conta de origem) nos últimos 90 dias. Valor igual ou maior, ou nova tentativa com 10 minutos ou mais, não dispara. Sem rejeição anterior a regra não atua (sem histórico, nunca é evidência). Vale para todos os fluxos.	`RETRY_AFTER_REJECTION_WINDOW_MINUTES` = 10	`RETRY_AFTER_REJECTION_WINDOW_MINUTES`: escolha de design, sem norma por trás; é o mesmo horizonte de 10 minutos da velocity (`VELOCITY_WINDOW_MINUTES`), curto o bastante para ligar a nova tentativa à rejeição sem pegar um cliente que desistiu e voltou depois. Reduzir o valor depois de uma rejeição é o comportamento de quem testa o limite do antifraude; a rejeição do motor é o rótulo aproximado de fraude que temos (não há confirmação externa), por isso o sinal é `suspicious` e não `critical`: dúvida de comportamento, que o 2FA resolve. Não foi calibrado ainda no simulador.
`HOW_TEST_PIX`	Eixo `how`, `suspicious`. É o "Pix de teste": o pagador mandou, nos últimos 15 minutos, um Pix de até R$ 10 (qualquer status) ao **mesmo destino**, e agora manda um Pix de pelo menos R$ 200 e de pelo menos 20 vezes o valor do teste a esse destino. Não dispara se o destino já era conhecido antes do teste, isto é, se existe Pix aprovado do pagador a esse destino anterior à janela de 15 minutos (o próprio Pix de teste, quando aprovado, não cria essa relação). O histórico (Pix pequeno no destino, já com o fator de 20 vezes aplicado ao valor atual) é calculado na consulta do resumo do pagador; o motor combina com o valor mínimo e com a relação anterior com o destino. Com os valores atuais, 20 vezes o teto de R$ 10 é exatamente o mínimo de R$ 200. Vale para todos os fluxos.	`TEST_PIX_MAX_AMOUNT_MINOR` = R$ 10 `TEST_PIX_WINDOW_MINUTES` = 15 `TEST_PIX_FOLLOWUP_MIN_AMOUNT_MINOR` = R$ 200 `TEST_PIX_FOLLOWUP_MULTIPLIER` = 20	`TEST_PIX_MAX_AMOUNT_MINOR`: escolha de design, sem norma por trás; golpista que testa se a conta recebe e se o dono não percebe costuma mandar valor simbólico, e R$ 10 deixa de fora Pix pequenos legítimos acima disso. `TEST_PIX_WINDOW_MINUTES`: escolha de design; o teste e o Pix grande vêm juntos, e 15 minutos cobre o intervalo de quem espera a confirmação antes de mandar o valor real. `TEST_PIX_FOLLOWUP_MIN_AMOUNT_MINOR`: escolha de design; abaixo de R$ 200 o Pix seguinte não compensa a investigação e coincide com a ordem de grandeza do piso de device não cadastrado. `TEST_PIX_FOLLOWUP_MULTIPLIER`: escolha de design; exigir um salto de 20 vezes separa o teste real do cliente que paga duas contas pequenas ao mesmo destino. É dúvida de comportamento (`how`) que o 2FA resolve, por isso `suspicious`. Não foi calibrado ainda no simulador.
`TRUST_RECURRENT_BENEFICIARY`	Sinal de confiança (eixo `trust`, sem nível). Dispara quando o pagador já tem **2 ou mais Pix aprovados** para a	`RECURRENT_BENEFICIARY_MIN_APPROVED` = 2	`RECURRENT_BENEFICIARY_MIN_APPROVED`: escolha de design, sem norma por trás. Dois pagamentos

Regra	Lógica	Parâmetros	Por quê
	mesma conta de destino **e** o primeiro deles foi aprovado **mais de 24h** antes do Pix em análise (24h exatas não bastam). Só conta `approved`; o dado vem da memória `account_counterparty` (consulta pontual pela chave do par, sem janela de 90 dias). Quando dispara, rebaixa **um** eixo `suspicious` para `ok`, na ordem who → how → where; nunca rebaixa um `critical` e nunca remove um piso regulatório. Se o rebaixamento muda a decisão, o `decision_basis` é `trust_downgrade` e o código aparece em `trust` no `metadata`.	`RECURRENT_BENEFICIARY_MIN_AGE_HOURS` = 24h	aprovados mostram uma relação anterior a este Pix; um só pode ser o próprio golpe que passou. `RECURRENT_BENEFICIARY_MIN_AGE_HOURS`: escolha de design. As 24h impedem o golpista de "fabricar" recorrência com dois Pix seguidos na mesma sessão: a relação precisa ter nascido antes do dia do ataque. Não foi calibrado ainda no simulador.
`TRUST_MATURE_SELF_PIX`	Sinal de confiança (eixo `trust`, sem nível). Dispara quando as duas pontas são do **mesmo titular** (`account_holder`), a conta de destino foi aberta há **30 dias ou mais** (29 não basta) **e** o device do Pix é `trusted`. Não exige histórico entre as duas contas. Mesmo efeito do destinatário recorrente: rebaixa **um** eixo `suspicious` (who → how → where), nunca um `critical`, nunca um piso.	`MATURE_SELF_PIX_MIN_DESTINATION_AGE_DAYS` = 30 dias	`MATURE_SELF_PIX_MIN_DESTINATION_AGE_DAYS`: escolha de design, sem norma por trás. O risco do self-Pix é a conta fantasma recém-aberta no nome da vítima; 30 dias é o mesmo corte de destino jovem de `WHERE_DESTINATION_NEW_ACCOUNT`, para as regras não se contradizerem. O device `trusted` garante que é o cliente de sempre movendo o próprio dinheiro. Exigir histórico entre as contas puniria o primeiro Pix legítimo para a própria conta antiga.

Como as regras viram decisão

Cada regra avaliada pontua um dos três eixos (`who`, `how`, `where`) com `ok`, `suspicious` ou `critical`. O nível consolidado do eixo é o mais severo entre suas regras.

A Matriz de Decisão

Situação dos Eixos	Decisão	Base (`decision_basis`)	Racional
Algum eixo `critical`	**`rejected`**	`critical_axis`	Fraude evidente (mula, golpe em massa, fundada suspeita); 2FA não resolve
2 ou mais eixos `suspicious`	**`rejected`**	`two_suspicious_axes`	Dúvidas combinadas (ex: noturno + conta nova); risco inaceitável
Exatamente 1 eixo `suspicious`	**`two_factor_required`**	`single_suspicious_axis`	Dúvida pontual de identidade ou comportamento sanável pelo titular
Todos os eixos `ok`	**`approved`**	`clean`	Transação alinhada ao perfil, sem anomalias detectadas

Pisos Regulatórios e Sinais de Confiança

- **Pisos Garantidos (`floors`):** Dispositivo sem histórico com valor > R$ 200 (`FLOOR_UNREGISTERED_DEVICE`) ou Pix noturno PF→PF > R$ 1.000 (`FLOOR_NIGHT_LIMIT` — Res. BCB nº 142/2021) impõem no mínimo `two_factor_required` (`regulatory_floor`), mesmo se os eixos estiverem `ok`. O piso nunca rejeita sozinho.
- **Sinais de Confiança (`trust`):** Destinatário recorrente (`TRUST_RECURRENT_BENEFICIARY`) ou self-Pix maduro (`TRUST_MATURE_SELF_PIX`) rebaixa **um** eixo `suspicious` para `ok` (na ordem de resolução: who → how → where). Sinais de confiança **nunca** rebaixam eixos críticos nem removem pisos regulatórios.
- **Cold Start Seguro:** Ausência de histórico **nunca** é evidência de fraude. A primeira transação de um cliente comum sem anomalias é aprovada (`clean`); ela só é rejeitada se o próprio payload contiver evidências em 2 eixos distintos ou se a evidência vier de entidade externa já comprometida (device com múltiplos CPFs ou conta recebedora sob fundada suspeita — Res. BCB nº 501/2025).

Exemplo do Desafio Oficial (Resultado Esperado: `rejected`)

Submetendo o payload de referência (`tests/fixtures/challenge_example_payload.json`): Pix PF→PF de R$ 1.500 às 22h15 para conta aberta há apenas 7 dias: 1. Eixo **HOW**: R$ 1.500 no período noturno excede o teto de R$ 1.000 → dispara `HOW_NIGHT_LIMIT_EXCEEDED` (`suspicious`). 2. Eixo **WHERE**: Destino aberto há 7,55 dias (< 30 dias) → dispara `WHERE_DESTINATION_NEW_ACCOUNT` (`suspicious`). 3. **Decisão:** 2 eixos suspeitos combinados na matriz → **`rejected`** com `decision_basis: "two_suspicious_axes"`.

Como rodar e testar

Toda requisição deve incluir o header `INTERNAL-TOKEN: internal-token-dev`.

1. Subir a API e o Banco

```
docker compose up -d --build
curl -s http://localhost:8001/health_check
# Resposta esperada: {"status":"ok"}
```

(API em `localhost:8001` e PostgreSQL em `localhost:5434`)

2. Testar o Payload do Desafio (POST /pix/transaction)

```
curl -s -w '\nHTTP %{http_code}\n' -X POST http://localhost:8001/pix/transaction \
  -H "Content-Type: application/json" \
  -H "INTERNAL-TOKEN: internal-token-dev" \
  -d @tests/fixtures/challenge_example_payload.json
```

Retorno esperado: `HTTP 201` com `status: "rejected"` e `decision_basis: "two_suspicious_axes"`.

Repetir a chamada com o mesmo ID devolve **`HTTP 200`** com o mesmo corpo (idempotência confirmada).

3. Consultar Análise (GET /pix/transaction/{id})

```
curl -s -H "INTERNAL-TOKEN: internal-token-dev" http://localhost:8001/pix/transaction/pix-000123
```

Retorna `HTTP 200` com a mesma análise imutável. IDs inexistentes retornam `404 RNV001039`.

4. Executar Testes Automatizados

Os testes de integração são caixa-preta: falam com a API em `http://localhost:8001` e conferem o banco em `localhost:5434`. Por isso a stack do passo 1 precisa estar de pé antes de rodar o `pytest`.

4.1. Criar e ativar o ambiente virtual (Python 3.11, a mesma versão da CI):

```
python3.11 -m venv .venv
source .venv/bin/activate          # Windows (PowerShell): .venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

4.2. Instalar as dependências de teste (`requirements-dev.txt` já inclui o `requirements.txt`, mais `pytest`, `pytest-cov`, `python-dotenv` e `flake8`):

```
pip install -r requirements-dev.txt
```

4.3. Garantir que a API e o banco estão no ar:

```
docker compose up -d --build
curl -s http://localhost:8001/health_check   # {"status":"ok"}
```

4.4. Rodar lint e testes (com o venv ativo, na raiz do repositório):

```
flake8
python -m pytest                          # suíte completa
python -m pytest tests/engine             # só o motor (função pura, não precisa da API)
python -m pytest tests/architecture       # só as regras de camadas (AST)
python -m pytest tests/integration        # só a integração HTTP (precisa da stack no ar)
python -m pytest --cov=src --cov-report=term-missing   # com cobertura
```

Executa a suíte de 367 testes (integração HTTP caixa-preta, testes de motor e regras de arquitetura por AST) em ~5 segundos, com 100% de sucesso e zero alertas de lint.

Os valores padrão já batem com o `docker-compose.yml`. Se a stack rodar em outra porta ou com outro token, exporte antes do `pytest`: `API_URL` (padrão `http://localhost:8001`), `INTERNAL_TOKEN` (padrão `internal-token-dev`), `DB_HOST`, `DB_PORT` (padrão `5434`), `DB_USER`, `DB_PASSWORD` e `DB_NAME` (padrões `rnv`, `rnv`, `antifraud`).

Os scripts dos passos 5 e 6 usam o mesmo venv (`python scripts/farm.py` e `python scripts/load_test.py`), também com a stack no ar.

5. Simulador de Tráfego e Personas (Farm)

Simula 7 personas pela própria API e valida o orçamento regulatório (0% de falsos positivos em transações legítimas):

```
python scripts/farm.py
```

```
================================================================================================================
  SIMULADOR DE PERSONAS E DESEMPENHO ANTIFRAUDE (FARM)
================================================================================================================
Policy Version:             2026.10.18-1
Transações Simuladas:       27
Distribuição Geral:         Aprovadas: 20 (74.1%) | 2FA: 3 (11.1%) | Rejeitadas: 4 (14.8%)
Orçamento Regulatório:      ATENDIDO (0% falsos positivos em legítimas)
----------------------------------------------------------------------------------------------------------------
--
TOTAL                       27      20 (74%)   3 (11%)  4 (15%)    [PASS]
================================================================================================================
```

6. Testes de Carga e Latência

Validação do SLA regulatório e arquitetural (< 100 ms) sob concorrência com banco ativo:

```
python scripts/load_test.py --requests 20 --concurrency 2
# Resultado medido: p50 = 42.3 ms | p95 = 84.8 ms (Meta < 100 ms ATENDIDA)
```

7. Coleção do Insomnia

Coleção completa pronta para importação em `insomnia/antifraud-pix.insomnia.json` (48 requisições cobrindo cenários de saúde, erro 400/403/422/409, decisões do motor e rajadas de carga).

Modelo de dados

O banco de dados PostgreSQL roda via script inicial `database/database.sql` estruturado em três grupos: - **Lookups:** Tabelas enumeradas (`analysis_status`, `decision_basis`, `risk_level`, `rule_axis`, `rule_outcome`, `pix_participant` com 469 ISPBs do STR). - **Registro Imutável:** `account_holder` (titular único por CPF/CNPJ), `account` (agência e conta vinculadas ao titular), `pix_transaction` (o Pix recebido), `pix_analysis` (a decisão 1:1 com chave pública UUID v4), `pix_analysis_axis` (3 linhas por análise) e `pix_analysis_rule` (todas as regras avaliadas com valores observados e limiares). - **Memória de Aprendizado:** `account_device` e `account_counterparty` (contadores atômicos de aprovações prévias, alimentados exclusivamente por transações `approved`).

Performance: Todas as buscas síncronas de histórico utilizam índices de cobertura (*covering indexes* com cláusula `INCLUDE`), garantindo **Index Only Scan (< 0.1 ms)** sem leitura de páginas físicas de heap.

Opções descartadas

Opção Descartada	Justificativa Técnica
Redis	Descartado para evitar consistência eventual, risco de split-brain e complexidade de infraestrutura. Índices B-Tree com `INCLUDE` no Postgres respondem em microssegundos.
Machine Learning Caixa-Preta	Descartado porque a regulamentação do BCB e o cliente exigem explicabilidade auditável (`metadata.reasons`). O modelo determinístico por eixos elimina deriva e alucinações.
Gravar Response Body	Descartado para eliminar duplicidade de dados. Como a análise é append-only e imutável, o DTO reconstrói a resposta idêntica em retentativas e consultas GET.
UUID como Chave Primária	Descartado. UUID v4 causa fragmentação de B-Tree e page splits. Usamos `BIGSERIAL` sequencial interno (8 bytes) e UUID v4 nativo exclusivamente como chave pública externa.

Limitações conhecidas

1. **Parâmetros sem calibração com dados reais de produção:** O simulador `scripts/farm.py` valida 7 personas sintéticas contra o orçamento regulatório; ajustes finos adicionais dependem de telemetria de tráfego de produção.
2. **Teto do cliente por fórmula de design:** A fórmula de encolhimento (1,5 × p95 dos últimos 90 dias mesclado com teto de referência da fase) é uma escolha de engenharia de risco.
3. **Agravantes de geolocalização e país do IP:** Geolocalização é armazenada mas não avaliada; país do IP por connector externo não foi integrado nesta entrega.
4. **Concorrência simultânea sem lock pessimista:** Para preservar latência (< 100 ms), transações que chegam no mesmo milissegundo leem histórico já confirmado. A idempotência é garantida atômica por constraint única.
5. **Participantes STR Estáticos:** O seed contém os 469 participantes do STR na data de 16/10/2026; novos participantes homologados pelo BCB acionam temporariamente o sinal `WHERE_UNKNOWN_PARTICIPANT`.

Contrato da API

Rota	Acesso	Sucesso	Erros Mapeados
`POST /pix/transaction`	Serviço (`INTERNAL-TOKEN`)	`201 {key, status, metadata}`; retry `200`	400 (`RNV000001`, `RNV000007`–`010`), 403 (`RNV000002`), 422 (`RNV001002`, `RNV001020`, `RNV001024`, `RNV001040`–`043`), 409 (`RNV001044`)
`GET /pix/transaction/{id}`	Serviço (`INTERNAL-TOKEN`)	`200 {key, status, metadata}`	403 (`RNV000002`), 404 (`RNV001039`)
`GET /health_check`	Público	`200 {"status":"ok"}`	—